

解密程式碼的骨骼：

深入理解 Tree-sitter 與其結構化本質

Laurence Chen
<https://replware.dev>



Laurence Chen / 陳家宏

- Clojure, dbt-Taipei 線下活動主辦人
- IT 顧問、講者、作家
- 從試算表到資料平台－重構資料工程的技術與團隊

從試算表到資料平台 重構資料工程的技術與團隊

▲ 9/19 博客來/天瓏書局 正式發售

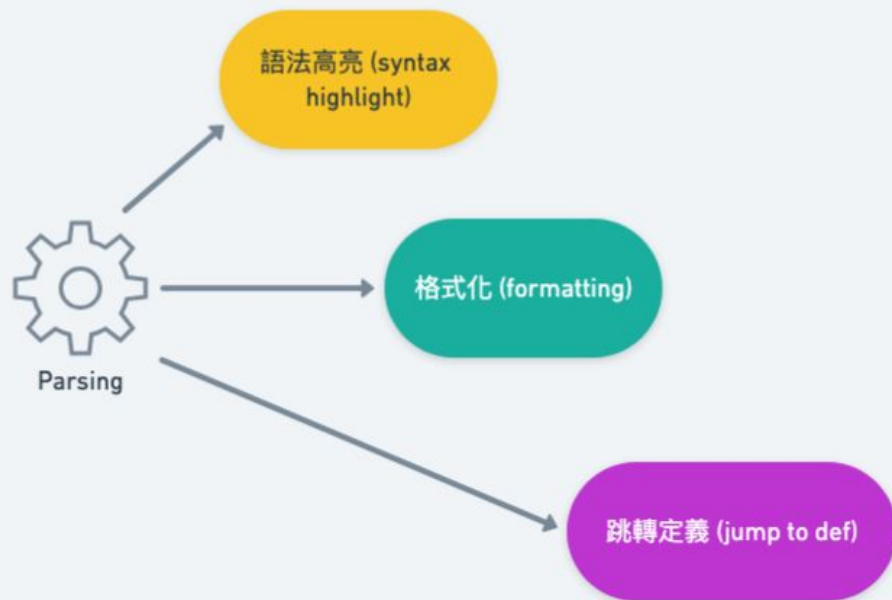


目錄

- 什麼是語法分析 (parse) ?
- 簡介 Tree-sitter
- 實際案例 (Neovim)
- 與 LLM 一起運用的潛力
- 結論

為什麼需要語法分析 (syntactic parsing) ?

- 語法高亮度
- 格式化
- 跳轉定義



Made with Whimsical

語法高亮度

- 當 type 也是一個函數名稱

```
(fn test-syntax-highlight [a type]  
  ;;  
)
```

```
(fn test-syntax-highlight [a type]  
  ;;  
)
```

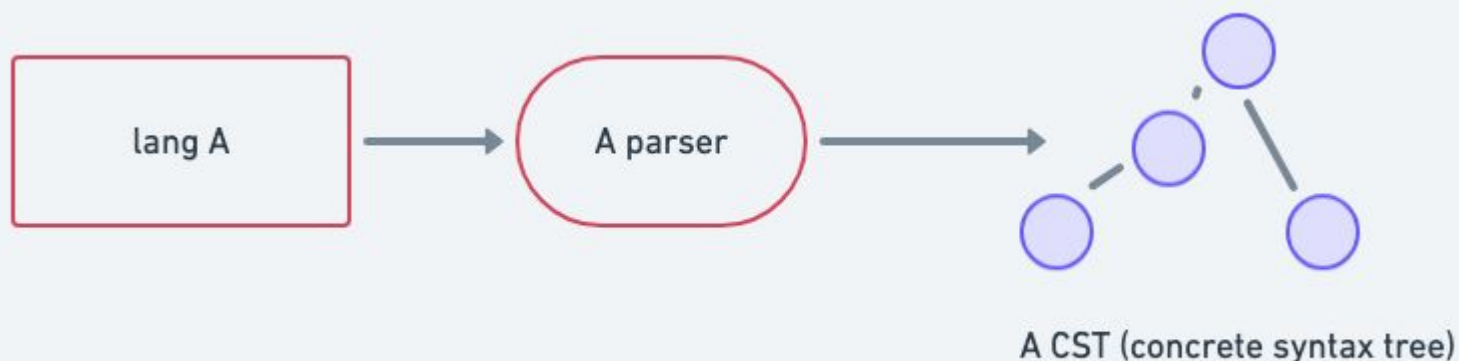
語法樹

```
(fn test-syntax-highlight [a type]
  ;;
)
```

```
(fn_form ; [177, 0] - [179, 3]
  call: (symbol) ; [177, 1] - [177, 3]
  name: (symbol) ; [177, 4] - [177, 25]
  args: (sequence_arguments ; [177, 26] - [177, 34]
    item: (symbol_binding) ; [177, 27] - [177, 28]
    item: (symbol_binding)) ; [177, 29] - [177, 33]
  (comment ; [178, 2] - [178, 5]
    body: (comment_body))) ; [178, 4] - [178, 5]
```

語法分析的挑戰

- 每種語言使用各自獨立的 parser
- 工具鏈不一致, 維護成本高
- 缺乏統一 API



Made with Whimsical

Tree-sitter

- 通用的 Parser Generator (解析器產生器)
- 特色
 - 增量解析 (Incremental Parsing)
 - 通用性與語言中立 (Universal & Language-Agnostic)
 - 容錯性 (error tolerance)

Tree-sitter (如何使用 – part 1)

- Setup

- [grammar.js](https://github.com/tree-sitter/grammar.js)
- Neovim
 - nvim-treesitter plugin
 - :TSInstall fennel

Tree-sitter (如何使用 – part 2)

- Lib API interface

```
;;TSQuery that matches `local, fn`  
(local def-query (vim-ts.query.parse :fennel  
                                "  
(local_form  
  (binding_pair  
    lhs: (symbol_binding) @local.def))  
(fn_form  
  name: [(symbol) (multi_symbol)] @fn.def)"))
```

Tree-sitter (如何使用 – part 3)

- Tooling

- Neovim/ Emacs/ VSCode

- :InspectTree

顯示 Tree-sitter 的 CST 在新視窗

- :EditQuery

提供一個視窗，讓使用者輸入 Query。

同時，對當前的 Query 可以查詢成功的程式碼區段，在 source code 視窗裡做出反白。

Tree-sitter (如何使用 – part 3)

- Tooling

- InspectTree
- EditQuery

```
(fn search-in-ext-buffer [code-text last-row bufnr] fn  
  "Search defs inside one buffer"  
  (let [curr-targets (search-ext-targets def-query (get-current-line)  
    results (core.filter (fn [node-t]  
      (= code-text node-t.content))  
      curr-targets)]  
    results)))  
  
(fn jump-to-range [range] fn  
  (vim.api.nvim_win_set_cursor 0 range.start))  
  
(fn rest-str [s] fn  
  "equal to clojure's (str (rest s))"  
  (string.sub s 2 -1))  
  
(fn resolve-lua-module-path [modname] fn  
  "Try to resolve a lua module via package.path"  
  (package.searchpath (.. "lua." modname) package.path))  
  
(fn resolve-fnl-module-path [modname] fn  
  "Try to resolve a fnl module via fennel.path"  
  (package.searchpath modname (. (config.default) :fennel-path)))
```

<nl/conjure/client/fennel/def-str-util.fnl 70,1 45%

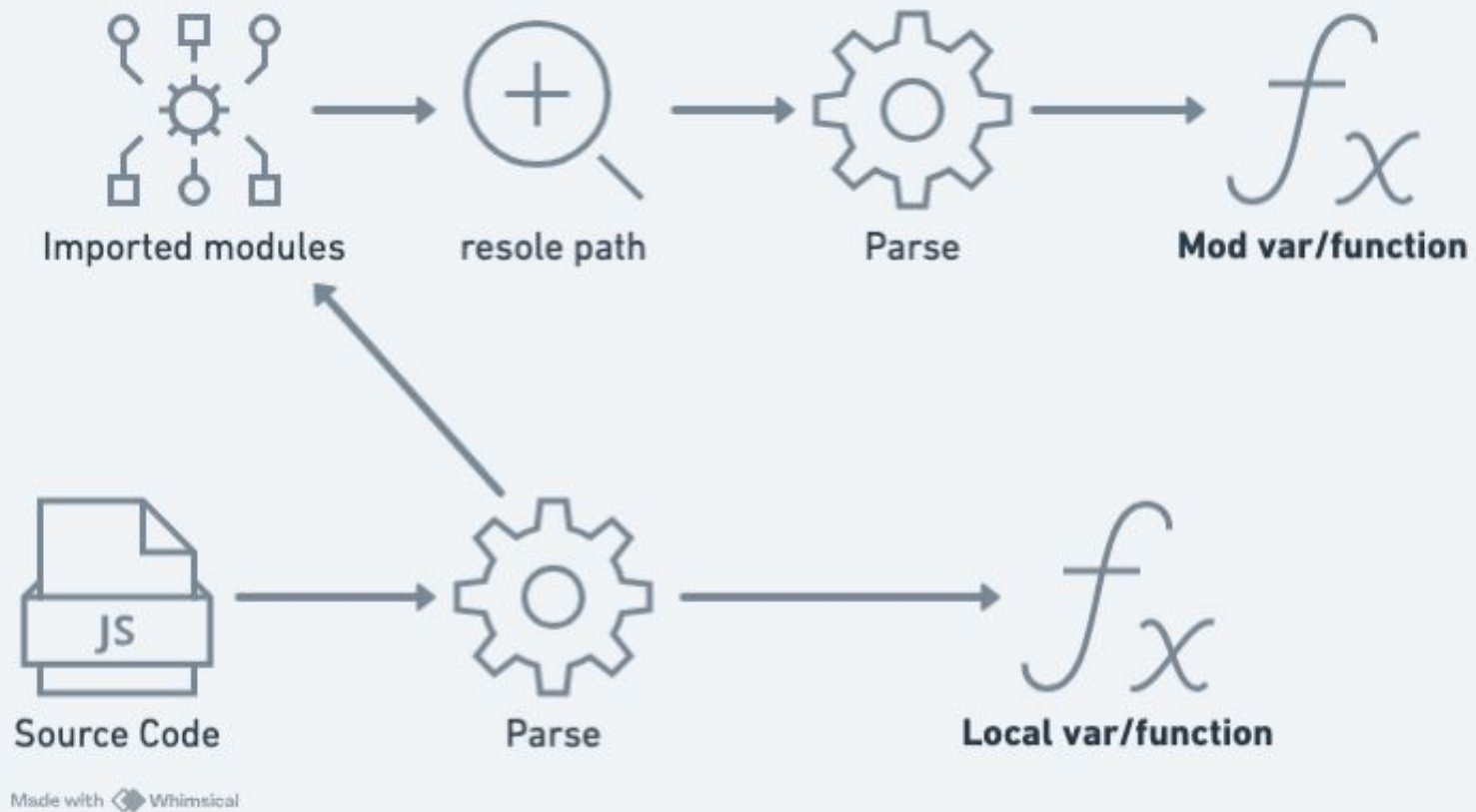
我的問題：在開發之中的某些無效率

- 在開發 Redux 的程式時, View 會呼叫 Action , 而我想要從 Action 去跳轉 Action 的 Reducer 。
- 在開發 dbt Model 時, a Model 會引用不同的 b Model , 而我想要從 a Model 直接跳轉 b Model 。
- 這類的問題很常見, 而且偶爾就是會有編輯器／插件不支援、很難灌、或是剛好有一些 bug 。

也許你不應該 grep ?

- 你是否正在找某個東西的定義？(Go to Definition)
- 你是否正在找某個東西的引用？(Go to Reference)

Workaround Design of jump to def



Workaround Design of jump to def

1. 解析本地檔案，找出所有的變數、函數的定義。
2. 解析本地檔案，找出所有的 Import Module，並且 resolve 為 Module 的定義檔案。最後，再將這些定義檔裡的變數、函數的定義都解析出來。
3. 參考：<https://github.com/Olical/conjure/pull/704>

思考：語言是結構化資料嗎？

s (句子)

├─ NP (名詞短語) : 我

├─ VP (動詞短語)

│ ├─ V (動詞) : 是

│ └─ PP (介詞短語)

│ ├─ 在 (介詞)

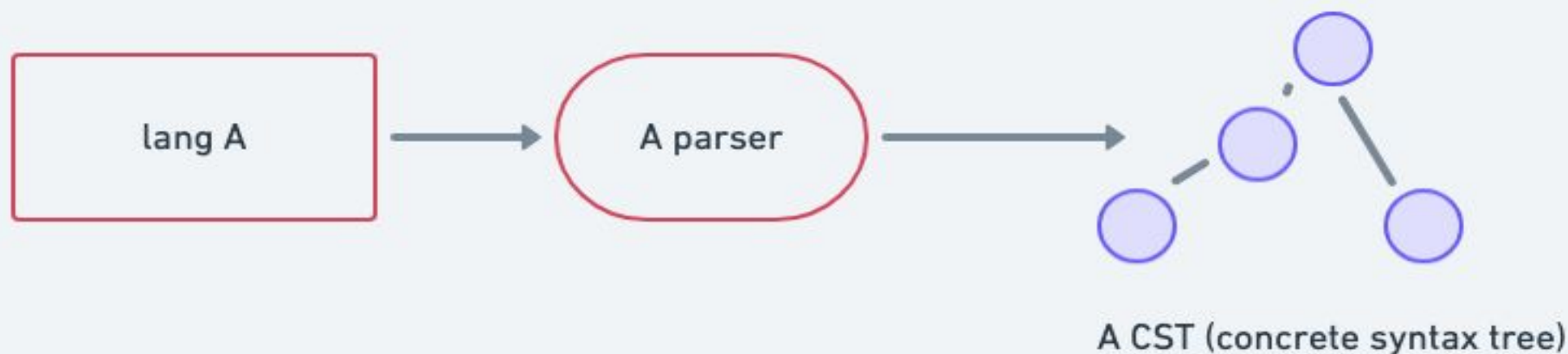
│ └─ VP (動詞短語)

│ ├─ V (動詞) : 浪費

│ └─ NP (名詞短語) : 我的時間

程式語言可以是「結構化資料」嗎？

- 試試用 ast-grep 來做重構！
- 如果我們把 CST/AST 與 source code 一同給予 LLM 做為 input 呢？



Made with Whimsical

Q & A

